

# Transferable Neural Networks for Partial Differential Equations

Zezhong Zhang, Feng Bao, Lili Ju, Guannan Zhang

## 1 Introduction

Partial differential equations (PDEs) are fundamental to modeling physical phenomena across various scientific disciplines. Traditional numerical methods for solving PDEs often require extensive computational resources, especially for complex geometries. While neural network-based methods have emerged as promising alternatives, they suffer from high computational costs associated with model training and retraining using stochastic gradient descent (SGD).

Transfer learning approaches for PDEs typically involve pre-training neural networks using either the PDE’s formulation or solution data. This limits their transferability to similar PDEs and requires generating training data using other PDE solvers. To overcome these limitations, this paper introduces Transferable Neural Networks (TransNet), which constructs a pre-trained neural feature space without using any PDE information, enabling the transfer of the pre-trained feature space to various PDEs with different domains and boundary conditions.

## 2 Transferable Neural Feature Space

### 2.1 Problem Setting

Consider a general PDE formulation:

$$\begin{cases} L(u(y)) = f(y) & \text{for } y \in \Omega \\ B(u(y)) = g(y) & \text{for } y \in \partial\Omega \end{cases} \quad (1)$$

where  $\Omega \subset \mathbb{R}^d$  is the spatial-temporal bounded domain,  $y = (x, t) \in \mathbb{R}^d$  includes both spatial and temporal variables,  $u$  is the unknown solution,  $L$

and  $B$  are differential operators defining the PDE and boundary conditions respectively, and  $f$  and  $g$  are the corresponding right-hand sides.

A single-hidden-layer fully-connected neural network representation of the solution is:

$$u_{NN}(y) := \sum_{m=1}^M \alpha_m \sigma(w_m^T y + b_m) + \alpha_0 \quad (2)$$

where  $M$  is the number of hidden neurons,  $w_m \in \mathbb{R}^d$  and  $b_m \in \mathbb{R}$  are the weights and bias of the  $m$ -th hidden neuron,  $\alpha = (\alpha_0, \alpha_1, \dots, \alpha_M)^T$  represents the output layer parameters, and  $\sigma(\cdot)$  is the activation function.

## 2.2 Neural Feature Space Construction

The neural feature space is defined as the linear space expanded by the basis functions:

$$\mathcal{P}_{NN} = \text{span}\{1, \sigma(w_1^T y + b_1), \dots, \sigma(w_M^T y + b_M)\} \quad (3)$$

The key innovation in TransNet is the construction of this feature space to be transferable across different PDEs. This construction involves three main steps:

### 2.2.1 Re-parameterization of Hidden Neurons

Each hidden neuron  $\sigma(w_m^T y + b_m)$  is re-parameterized to separate the location and shape parameters:

$$\sigma(w_m^T y + b_m) = \sigma(\gamma_m(a_m^T y + r_m)) \quad (4)$$

where  $\|a_m\|_2 = 1$  and  $\gamma_m \geq 0$ . The equivalence is given by:

$$\begin{cases} w_m = \gamma_m a_m \\ b_m = \gamma_m r_m \end{cases} \Leftrightarrow \begin{cases} a_m = \frac{w_m}{\|w_m\|_2} \\ r_m = \frac{b_m}{\|w_m\|_2} \\ \gamma_m = \|w_m\|_2 \end{cases} \quad (5)$$

After re-parameterization, each parameter has a distinct geometric interpretation: -  $a_m$  represents the normal direction of the partition hyperplane  $a_m^T y + r_m = 0$  -  $|r_m|$  indicates the distance of the hyperplane from the origin -  $\gamma_m$  controls the steepness of the transition from -1 to 1 in the activation function

### 2.2.2 Generating Uniformly Distributed Neurons

To create a feature space with uniformly distributed neurons, the location parameters  $(a_m, r_m)$  are generated by:

$$a_m = \frac{X_m}{\|X_m\|_2} \quad \text{and} \quad r_m = U_m, m = 1, \dots, M \quad (6)$$

where  $X_m$  are i.i.d. from standard Gaussian distribution and  $U_m$  follows i.i.d. uniform distribution on  $[0, 1]$ .

The uniformity of neuron distribution is measured by the partition hyperplane density  $D_M^\tau(y)$ , defined as:

$$D_M^\tau(y) = \frac{1}{M} \sum_{m=1}^M 1_{\{d_m(y) < \tau\}}(y) \quad (7)$$

where  $d_m(y) = |a_m^T y + r_m|$  is the distance from  $y$  to the  $m$ -th partition hyperplane, and  $\tau > 0$  is the bandwidth for density calculation.

### 2.2.3 Tuning Shape Parameters Using Auxiliary Functions

The shape parameters  $\gamma_m$  are tuned to optimize the expressive power of the feature space. For simplicity, all neurons use the same shape parameter  $\gamma$ . To find the optimal  $\gamma$ , the method uses realizations of Gaussian random fields (GRFs) as auxiliary functions.

For  $K$  realizations of a GRF, the mean squared error (MSE) for each realization is:

$$\text{MSE}_k(\gamma) = \min_{\alpha} \sum_{j=1}^J \left[ g_j^k - \left( \sum_{m=1}^M \alpha_m \sigma(\gamma a_m y_j^k + \gamma r_m) + \alpha_0 \right) \right]^2 \quad (8)$$

where  $g_j^k = G(y_j | \omega_k, \eta)$  is the value of the  $k$ -th GRF realization at sample  $y_j^k$ , and  $\eta$  is the correlation length.

The optimal  $\gamma$  minimizes the average MSE:

$$\gamma_{opt} = \arg \min_{\gamma} \frac{1}{K} \sum_{k=1}^K \text{MSE}_k(\gamma) \quad (9)$$

---

**Algorithm 1** Tuning shape parameter  $\gamma$ 

---

**Input:** Number of basis functions  $M$ ; correlation length  $\eta$ ; number of GRF realizations  $K$ ; grid search range  $(\gamma_{min}, \gamma_{max})$  and mesh size  $S$

**Output:** Optimal shape parameter  $\gamma_{opt}$

**for**  $k = 1$  to  $K$  **do**

    Sample  $\{y_j^k\}_{j=1}^J$  uniformly in the unit ball

    Evaluate a realization of GRF:  $\{g_j^k = G(y_j|\omega_k, \eta)\}_{j=1}^J$

**end for**

Generate uniform mesh for  $\gamma$ :  $\gamma_{min} = \gamma_1 < \dots < \gamma_S = \gamma_{max}$

**for**  $s = 1$  to  $S$  **do**

**for**  $k = 1$  to  $K$  **do**

        Set  $\psi_0(y) = 1$

**for**  $m = 1$  to  $M$  **do**

            Generate shape parameter  $(a_m, r_m)$  according to Eq. (10)

            Calculate weight and bias:  $w_m = \gamma_s a_m$ ,  $b_m = \gamma_s r_m$

            Construct neural basis function:  $\psi_m(y) = \sigma(w_m^T y + b_m)$

**end for**

        Compute least square MSE:  $\text{MSE}_{s,k} = \min_{\alpha} \sum_j [g_j^k - \sum_{m=0}^M \alpha_m \psi_m(y_j^k)]^2$

**end for**

$\text{avgMSE}_s = \frac{1}{K} \sum_{k=1}^K \text{MSE}_{s,k}$

**end for**

Find the smallest  $\text{avgMSE}_s$ :  $s^* = \arg \min_{s=1}^S \text{avgMSE}_s$

**return**  $\gamma_{opt} = \gamma_{s^*}$ 

---

### 3 Theoretical Properties

#### 3.1 Uniform Neuron Distribution

A key theoretical result establishes that the proposed method for generating location parameters results in uniformly distributed neurons:

[Uniform neuron distribution] Given the partition hyperplane defined by  $a_m^T y + r_m = 0$ , if  $\{a_m\}_{m=1}^M$  are i.i.d. and uniformly distributed on the  $d$ -dimensional unit sphere, and  $\{r_m\}_{m=1}^M$  are i.i.d. and uniformly distributed in  $[0, 1]$ , then, for a fixed  $\tau \in (0, 1)$ ,

$$E[D_M^\tau(y)] = \tau \quad \text{for all } \|y\|_2 \leq 1 - \tau \quad (10)$$

where  $D_M^\tau(y)$  is the density function defined earlier.

This theorem guarantees that the expected density of partition hyperplanes is constant throughout the computational domain, which ensures uniform expressive power across the domain.

### 4 Applying TransNet to PDEs

#### 4.1 Linear PDEs

For linear PDEs, after constructing the neural feature space  $\mathcal{P}_{NN}$ , the solution is found by solving the least squares problem:

$$\min_{\alpha} \left\{ \frac{1}{J_1} \sum_{j_1=1}^{J_1} \left[ f(y_{j_1}^{PDE}) - \sum_{m=0}^M \alpha_m L(\psi_m(y_{j_1}^{PDE})) \right]^2 + \frac{1}{J_2} \sum_{j_2=1}^{J_2} \left[ g(y_{j_2}^{BD}) - \sum_{m=0}^M \alpha_m B(\psi_m(y_{j_2}^{BD})) \right]^2 \right\} \quad (11)$$

where  $\{y_{j_1}^{PDE}\}_{j_1=1}^{J_1}$  and  $\{y_{j_2}^{BD}\}_{j_2=1}^{J_2}$  are samples from  $\Omega$  and  $\partial\Omega$  respectively.

#### 4.2 Nonlinear PDEs

For nonlinear PDEs, TransNet can be combined with established nonlinear iterative solvers like Picard's method. In each iteration, the PDE is linearized, and the coefficients  $\alpha$  are updated by solving a least squares problem.

### 5 Numerical Examples and Results

Extensive numerical experiments were conducted to evaluate the performance of TransNet across various PDEs:

---

**Algorithm 2** Solving linear PDE with TransNet

---

**Input:** Number of basis functions  $M$ ; samples from  $\Omega$ ,  $\{y_{j_1}^{PDE}\}_{j_1=1}^{J_1}$ ; samples from  $\partial\Omega$ ,  $\{y_{j_2}^{BD}\}_{j_2=1}^{J_2}$

**Output:**  $\alpha_{OPT}$

Get  $\gamma_{OPT}$  for  $M$  from the offline tuning result

Set  $\psi_0(y) = 1$

**for**  $m = 1$  to  $M$  **do**

    Generate shape parameter  $(a_m, r_m)$  according to Eq. (10)

    Calculate weight and bias:  $w_m = \gamma_{OPT}a_m$ ,  $b_m = \gamma_{OPT}r_m$

    Construct neural basis function:  $\psi_m(y) = \sigma(w_m^T y + b_m)$

**end for**

Evaluate PDE feature  $F^{PDE} \in \mathbb{R}^{J_1 \times (M+1)}$ :  $F_{i,j}^{PDE} = L\psi_{j-1}(y_i^{PDE})$

Evaluate PDE target  $T^{PDE} \in \mathbb{R}^{J_1 \times 1}$ :  $T_i^{PDE} = f(y_i^{PDE})$

Evaluate boundary feature  $F^{BD} \in \mathbb{R}^{J_2 \times (M+1)}$ :  $F_{i,j}^{BD} = B\psi_{j-1}(y_i^{BD})$

Evaluate boundary target  $T^{BD} \in \mathbb{R}^{J_2 \times 1}$ :  $T_i^{BD} = g(y_i^{BD})$

Solve least squares problem:  $\alpha_{OPT} = \arg \min_{\alpha} \left\| \begin{bmatrix} F^{PDE} \\ F^{BD} \end{bmatrix} \alpha - \begin{bmatrix} T^{PDE} \\ T^{BD} \end{bmatrix} \right\|_2^2$

**return**  $\alpha_{OPT}$

---

1. Poisson equation in different domains: - 2D box domain - 2D circular domain - 2D L-shaped domain - 2D annulus domain - 3D box domain

2. Steady-state Navier-Stokes equation (2D)

3. Time-dependent PDEs: - Fokker-Planck equation (1D space + 1D time) - 2D Fokker-Planck equation (2D space + 1D time) - 1D wave equation (1D space + 1D time)

TransNet was compared with two baseline methods: random feature models and Physics-Informed Neural Networks (PINN). All methods used the same network architecture with tanh activation.

## 5.1 Feature Space Construction

Tests on the construction of transferable neural feature spaces in 2D and 3D unit balls showed that:

1. The optimization landscape for tuning the shape parameter  $\gamma$  resembles a parabolic curve, making it easy to find the optimal  $\gamma$ .

2. The optimal value for  $\gamma$  varies with the number of hidden neurons, confirming the necessity of tuning  $\gamma$  when changing network size.

3. The MSE distribution when using TransNet to approximate Gaussian

processes is uniform across the domain, demonstrating the effectiveness of the uniform neuron distribution.

## 5.2 PDE Solving Performance

The numerical results showed that TransNet significantly outperforms the baseline methods in several aspects:

1. **Transferability**: TransNet successfully solved various PDEs in different domains using only one 2D feature space and one 3D feature space, without retraining.
2. **Accuracy**: TransNet achieved several orders of magnitude smaller MSE than PINN and random feature models across all test cases. This advantage grows as the number of hidden neurons increases.
3. **Efficiency**: TransNet is computationally efficient, with training times comparable to random feature models and significantly faster than PINN, which requires SGD-based training.
4. **Neuron Distribution**: The density function analysis showed that while TransNet maintains uniform neuron density throughout the domain, both PINN and random feature models have highly non-uniform densities, with higher concentration near the center of the domain. This explains their inferior performance.

## 6 Implications and Advantages

The TransNet approach offers several significant advantages for solving PDEs:

1. **Independence from PDE Information**: Unlike existing transfer learning approaches, TransNet constructs a feature space without using any specific PDE information, making it truly transferable across different PDEs.
2. **Avoidance of SGD-Based Training**: By fixing the feature space and only solving for the output layer weights, TransNet avoids the drawbacks of SGD-based training, including slow convergence and ill-conditioning.
3. **Exploitation of Expressive Power**: The uniform distribution of neurons ensures equal expressive power across the entire domain, enabling TransNet to better capture solution features throughout the domain.
4. **Computational Efficiency**: The online operation for solving PDEs only requires solving a single least squares problem, which can be done efficiently using standard solvers.
5. **Robust Performance**: TransNet maintains its accuracy advantage across different types of PDEs (linear, nonlinear, steady-state, time-dependent) and different domain shapes.

These properties make TransNet a promising approach for practical applications where computational efficiency and solution accuracy are crucial considerations.